

Extended Euclidean Algorithm

[Fermat's theorem](#) allows us to calculate modular multiplicative inverses through [binary exponentiation](#) in $O(\log n)$ operations, but it only works with prime moduli. There is a generalization of it, [Euler's theorem](#), stating that if m and a are coprime, then

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

where $\phi(m)$ is [Euler's totient function](#) defined as the number of positive integers $x < m$ that are coprime with m . In the special case when m is a prime, then all the $m - 1$ residues are coprime and $\phi(m) = m - 1$, yielding the Fermat's theorem.

This lets us calculate the inverse of a as $a^{\phi(m)-1}$ if we know $\phi(m)$, but in turn, calculating it is not so fast: you usually need to obtain the [factorization](#) of m to do it. There is a more general method that works by modifying the [the Euclidean algorithm](#).

Algorithm

Extended Euclidean algorithm, apart from finding $g = \gcd(a, b)$, also finds integers x and y such that

$$a \cdot x + b \cdot y = g$$

which solves the problem of finding modular inverse if we substitute b with m and g with 1:

$$a^{-1} \cdot a + k \cdot m = 1$$

Note that, if a is not coprime with m , there is no solution since no integer combination of a and m can yield anything that is not a multiple of their greatest common divisor.

The algorithm is also recursive: it calculates the coefficients x' and y' for $\gcd(b, a \bmod b)$ and restores the solution for the original number pair. If we have a solution (x', y') for the pair $(b, a \bmod b)$

$$b \cdot x' + (a \bmod b) \cdot y' = g$$

then, to get the solution for the initial input, we can rewrite the expression $(a \bmod b)$ as

$(a - \lfloor \frac{a}{b} \rfloor \cdot b)$ and substitute it into the aforementioned equation:

$$b \cdot x' + (a - \lfloor \frac{a}{b} \rfloor \cdot b) \cdot y' = g$$

Now we rearrange the terms grouping by a and b to get

$$a \cdot \underbrace{y'}_x + b \cdot \underbrace{(x' - \lfloor \frac{a}{b} \rfloor \cdot y')}_y = g$$

Comparing it with the initial expression, we infer that we can just use coefficients of a and b for the initial x and y .

Implementation

We implement the algorithm as a recursive function. Since its output is not one but three integers, we pass the coefficients to it by reference:

```
int gcd(int a, int b, int &x, int &y) {
    if (a == 0) {
        x = 0;
        y = 1;
        return b;
    }
    int x1, y1;
    int d = gcd(b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return d;
}
```

To calculate the inverse, we simply pass a and m and return the x coefficient the algorithm finds. Since we pass two positive numbers, one of the coefficient will be positive and the other one is negative (which one depends on whether the number of iterations is odd or even), so we need to optionally check if x is negative and add m to get a correct residue:

```
int inverse(int a) {
    int x, y;
    gcd(a, M, x, y);
    if (x < 0)
        x += M;
    return x;
}
```

It works in $\sim 160\text{ns}$ — 10ns faster than inverting numbers with [binary exponentiation](#). To optimize it further, we can similarly turn it iterative — which takes 135ns :

```
int inverse(int a) {  
    int b = M, x = 1, y = 0;  
    while (a != 1) {  
        y -= b / a * x;  
        b %= a;  
        swap(a, b);  
        swap(x, y);  
    }  
    return x < 0 ? x + M : x;  
}
```

Note that, unlike binary exponentiation, the running time depends on the value of a . For example, for this particular value of m ($10^9 + 7$), the worst input happens to be 564400443 , for which the algorithm performs 37 iterations and takes 250ns .

Exercise. Try to adapt the same technique for the [binary GCD](#) (it won't give performance speedup though unless you are better than me at optimization).